

An Introduction to XProc 3.0

November 5, 2019

[Erik Siegel](#) ([siegels@cs.cmu.edu](#))

Erik Siegel is a senior research scientist at the MIT Computer Science and Artificial Intelligence Laboratory, where he works on the design and implementation of systems for processing large volumes of data in pipelines.

Table of Contents

[Introduction](#)

[What is XProc?](#)

[A first example](#)

[XProc Fundamentals](#)

[Pipelines or Steps?](#)

[Documents](#)

[Steps](#)

[Port Connections](#)

[Programming in XProc](#)

[Additional information](#)

Introduction

XProc is an XML-based language for processing documents in conversions and other tasks. It is the result of a series of efforts since 2010.

In 2017, a group of people (including Greg Ewing, Norman Walsh, and Erik Siegel) wrote a proposal (XProc 3.0) to the W3C. The proposal is a simplified and more powerful version of XProc 1.0. The latest version of the specification is in the making.

In November 2019, two drafts were released. The first draft is a standard around the end of 2019. The second draft is a more advanced and, most importantly, it is a more powerful language. The thing that is almost finished is the core of the language. The rest of the information is still in the early stages of development.

This site is the first of a 10-part series on XPath concepts.

What is XProc?

Let's answer this question with a high-level overview.

- XProc is a programming language defined in XML and written in XML.
- An XProc pipeline takes data as its input (often XML) and produces results.
- Steps are simple enough, like writing a program, splitting / combining input/output, or doing calculations, etc.
- Within a pipeline, everything is based on the data flowing through it.
- XProc pipelines are built by connecting steps.
- XProc also allows combining these steps in a just a few ways. Some steps can be selected or not.
- XProc also allows for conditional execution, looping, and other things.
- There is software that executes these pipelines, called processors.

Now why and when would this be useful? In the past, steps were often done in a sequence, one after another, and often in a loop. This is often done in a sequence, one after another, and often in a loop. This is often done in a sequence, one after another, and often in a loop.

A class of problems that can be solved by XProc is the problem of transforming XML. Some examples of this are: transforming XML to HTML, transforming XML to JSON, etc.

So why would we do this in the world of XML? One reason is that data is often in XML format, and it is often useful to transform it into a different format.

- We use XML to store data about our website.
- Multiple websites can share data, and it is often useful to transform it into a different format.

- Word processors create zip files with lots of XML documents and you need the text in order to read them. XML documents are not always wanted.

For as transformation of XML documents to other formats is often more than a simple task, it can be a complex one. The point of view is that the XML documents are not always wanted. The point of view is that the XML documents are not always wanted.

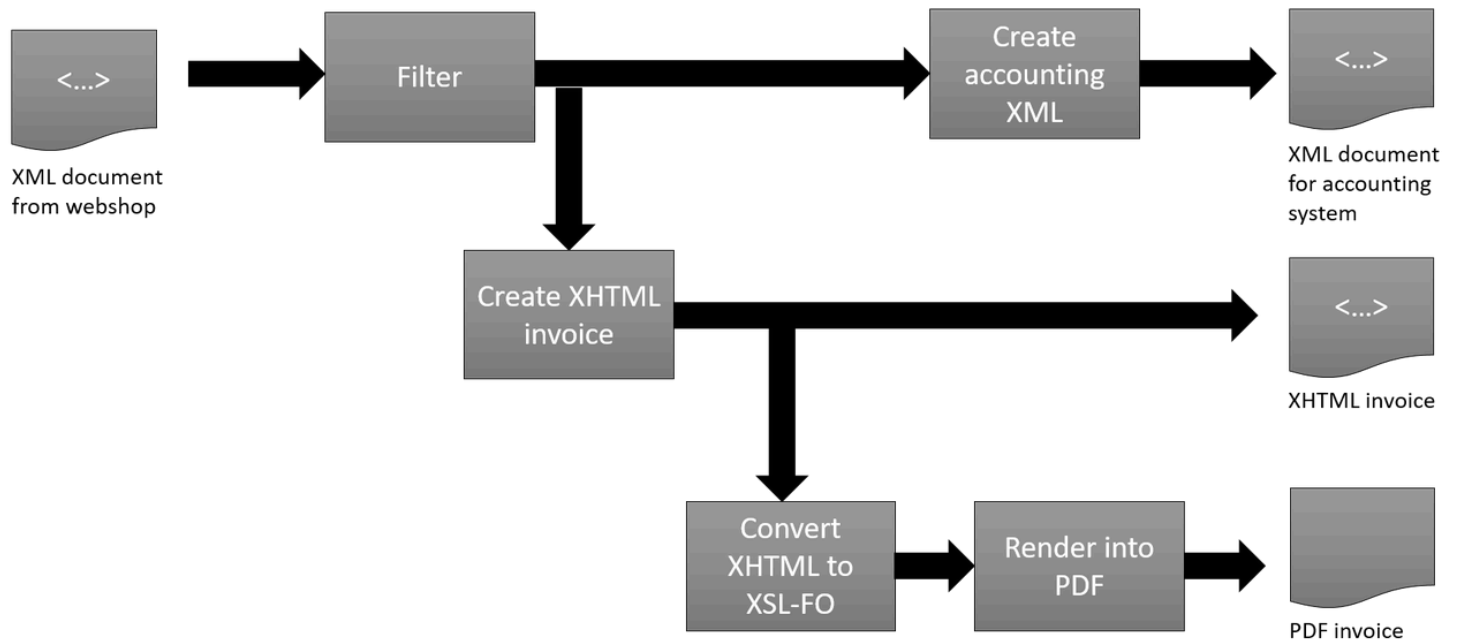
A first example

As a first simple example, consider the following application. We have a simple XML document that contains a list of items. We want to transform this XML document into a text file. This is a simple task, but it is not always wanted.

So we take the following XML document and:

- Filter the input document to get the items.
- Take the items and transform them into a text file.
- Create a new XML document from the text file.
- Create a new XML document from the text file.

In this example, we have a simple XML document that contains a list of items. We want to transform this XML document into a text file. This is a simple task, but it is not always wanted.



.(/media/images/pipeline - 1 . width - 1280 . png)

Figure 1: An example pipeline

And this is how it would look in XProc

Example 1. The example pipeline, expressed in XProc

;
; " OQN NF9 "

```

;,,
;,,  ANCX9  ,,

```

```

;,, rtaohodkhmd enq bqd shmf sgd WGS� hmunhbd9 ,,

```

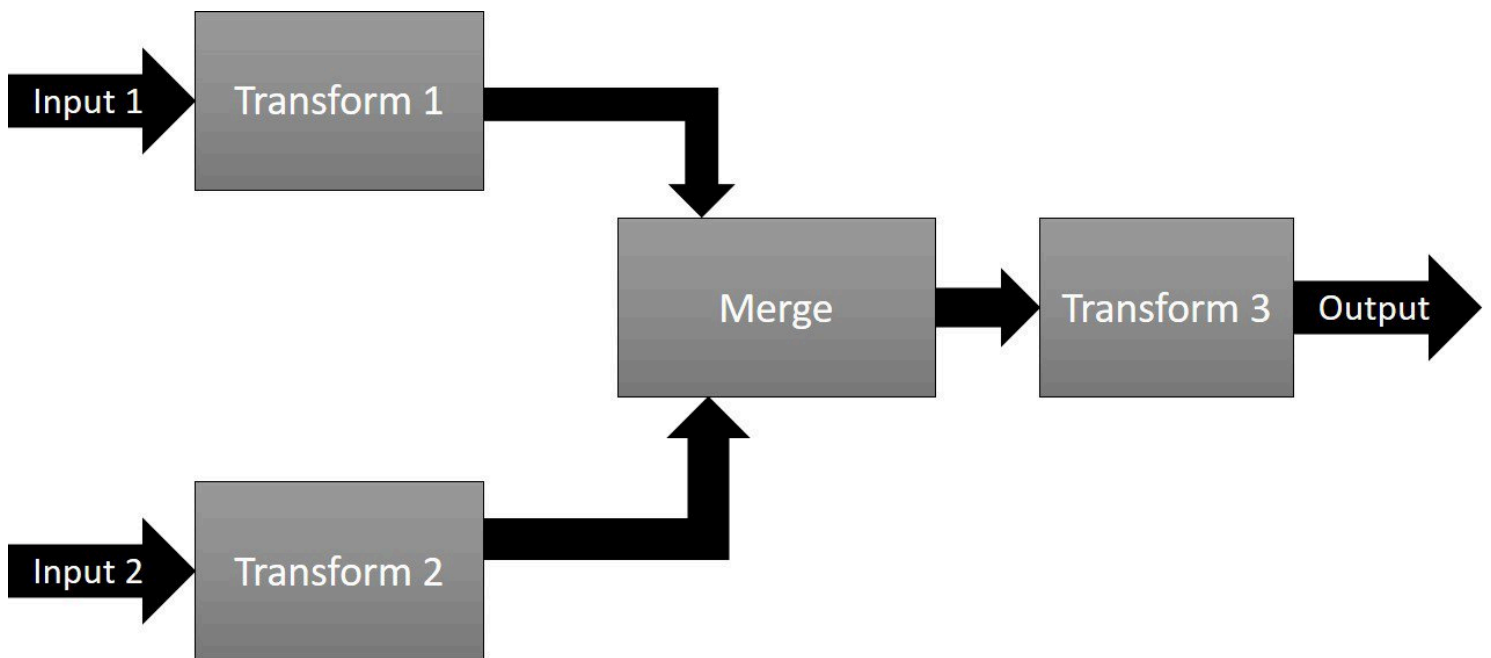
;,, rtaohodkhmd enq bqd shmf sgd OCE hmunhbd9 ,,

As very programming languages, in this sort of workflow, it is as easy as possible.

XProc Fundamentals

Pipelines or Steps?

XProc is a pipeline language. A pipeline is a series of steps connected with them using a series of output and input ports.



(/media/images/pipeline-with-steps.width-1280.jpg)

Figure 2: A pipeline containing steps

Steps take building blocks as input(s), data flowing and produces output(s). Now that does it? on the outside a step is a black box, but inside it is a computer, output processing being done inside and considered as a unit. We know then define

And that's exactly how XProc works. The two concepts are pipeline building blocks. One is a pipeline, the other is a step. The standard in XProc is a pipeline, and the other is a step.

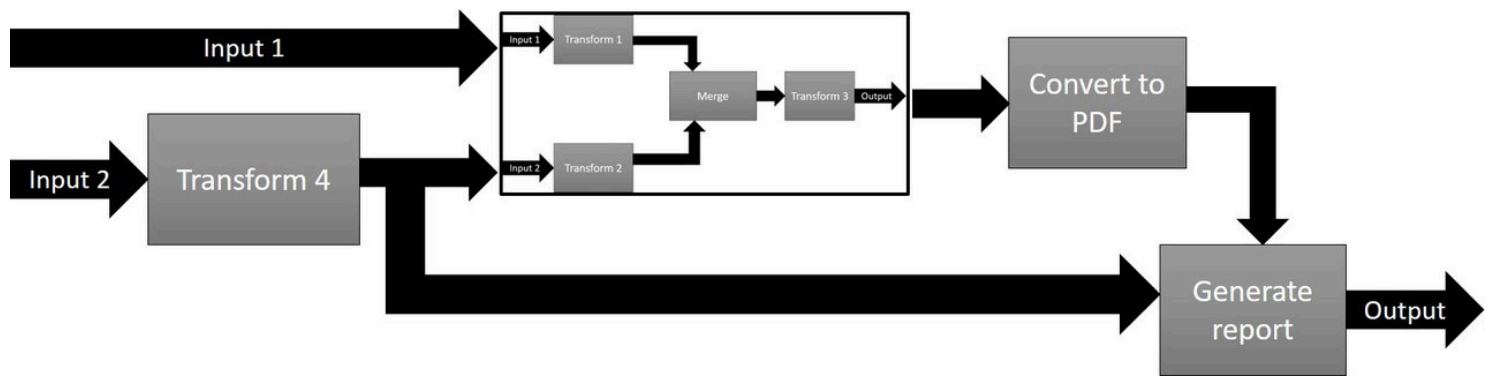


Figure 3: A pipeline using another pipeline as a step

Documents

Flowing through a pipeline, documents are transformed and then output. Although XML is a standard format for documents, it is not the only one. There are many other formats, and each has its own set of rules and conventions. The main document types are:

XML

Given its origin, it is no surprise that XML is designed for manipulating XML. It is a very flexible format, and it can be used to represent almost any type of data. However, it is not the most efficient format for representing data. It is a very verbose format, and it is not easy to read. It is a very powerful format, and it can be used to represent almost any type of data. It is a very flexible format, and it can be used to represent almost any type of data. It is a very powerful format, and it can be used to represent almost any type of data.

HTML

HTML documents are a subset of XML documents. They are designed for representing web pages. They are a very simple format, and they are easy to read. They are a very powerful format, and they can be used to represent almost any type of data. They are a very flexible format, and they can be used to represent almost any type of data. They are a very powerful format, and they can be used to represent almost any type of data.

JSON

Yes, there is a similarity between XML and JSON. They are both designed for representing data. They are a very simple format, and they are easy to read. They are a very powerful format, and they can be used to represent almost any type of data. They are a very flexible format, and they can be used to represent almost any type of data. They are a very powerful format, and they can be used to represent almost any type of data.

Text

Text files are a very simple format. They are designed for representing text. They are a very simple format, and they are easy to read. They are a very powerful format, and they can be used to represent almost any type of data. They are a very flexible format, and they can be used to represent almost any type of data. They are a very powerful format, and they can be used to represent almost any type of data.

Others

XML can also handle other types of data. It is a very flexible format, and it can be used to represent almost any type of data. It is a very powerful format, and it can be used to represent almost any type of data. It is a very flexible format, and it can be used to represent almost any type of data. It is a very powerful format, and it can be used to represent almost any type of data.

An interesting feature of XPath document handling is that it represents the document as a tree of nodes. The document will be represented as a pair of `XPath` objects: some of these are parts of the original document (the base URI and its sum of all the nodes) and can be inspected through the `XPath` pipeline.

Steps

XPort's main building is composed of two parts: the main part and the side part. The main part is a large building with a lot of windows. The side part is a smaller building with a lot of windows. The main part is a large building with a lot of windows. The side part is a smaller building with a lot of windows.

Type

All statements are a reflection of a foundation, or a programming language. The type: xslt is a transformation, or a piece of, XML inside an

To ~~ok~~ ~~eva~~ ~~step~~ ~~in~~ ~~pu~~ ~~se~~ ~~the~~ ~~type~~ ~~as~~ ~~the~~ ~~name~~ ~~and~~ ~~of~~ ~~pipeline~~ ~~is~~ ~~an~~ ~~val~~ ~~u~~ ~~e~~ ~~s~~ ~~formation~~ ~~:~~ ~~us~~ ~~ing~~ ~~the~~

Example 2. Invoking an XSLT transformation using the `p : x s` step

Ports

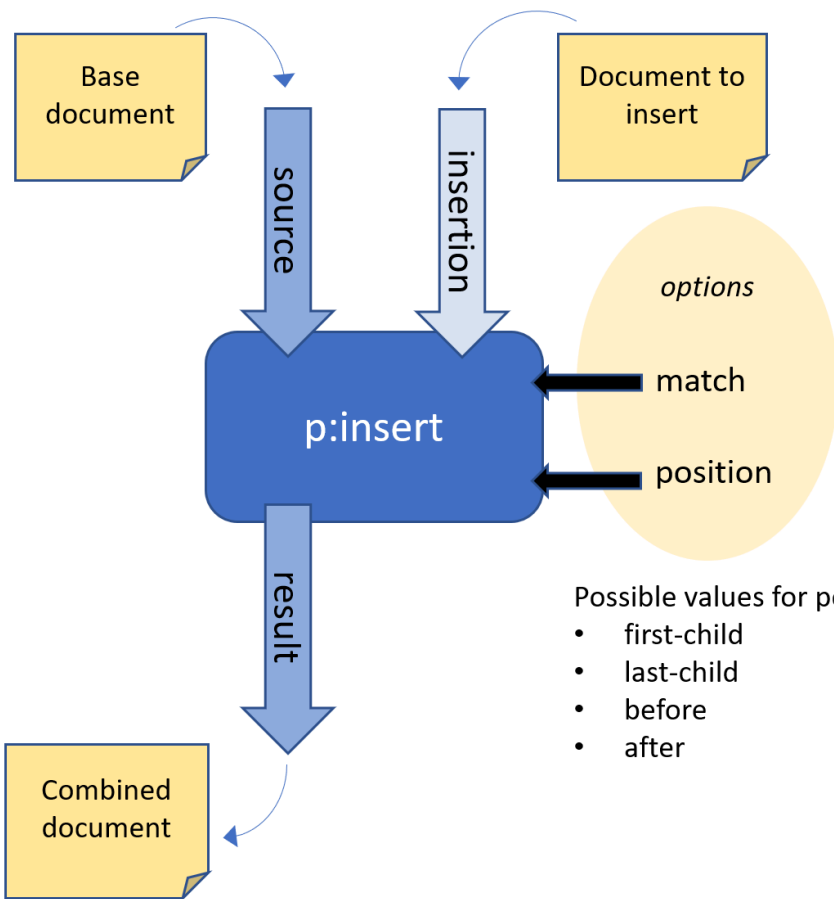
Port se a the o o n e f t a e s t e p u m v e l m e s f o t w s) i n o r (i o n u p t u s t) o p i
 Port se a the a b e n u t i v o f t h e r e U S o u r c y o n m e p c u t t e r o r t o h r e s n o e r
 y o u o u t e r

Aptomus set ~~name~~.eo You use this ~~same~~ ~~the~~ ~~op~~ pipeline.

Options

Steps ~~capth.a~~ ~~Optéon~~ ~~an~~ ~~ad~~ ~~ive~~ pairs t h o a t f u c e a r n d e e t e u r s n
s t e p f u n c t i o n a l i t y

As an example, let's take the following XML document:



(/media/image/step280.png)

Figure 4: A schematic representation of the `p:insert` step

- It has three ports:
 - 1 An input port named `source` which the base document flows in
 - 2 Another input port named `insertion` through which the document to be inserted flows in
 - 3 An output port named `result` that emits the combined document
- It has two options:
 - 1 An optional attribute named `match` which determines the main document the inserted document is matched against (e.g. `/r.doc // sect1`)
 - 2 An optional attribute named `position` which determines the position of the inserted document relative to the main document (e.g. `first-child`, `last-child`, `before` or `after`).

PortConnections

To make pipelines work, each step must have a way to connect its output to the input of the next step. In the case of the `p:insert` step, the output is the combined document, which is then passed to the next step in the pipeline.

Implicit binding

An implicit transformation specifies that one input feeds one output. The primary output of a transformation is the output that is used by the next transformation in the pipeline. When you place steps in a pipeline, the XSL processor implicitly connects them. Not only does implicit connection also connect the output of a transformation to the input of the next transformation. Here are the transformations:



(/media/images/implicit-connections.width-128

Figure 5: Implicit connections in a pipeline

Implicit binding is also important to understand.

Explicit binding

Besides implicit binding, you can also use explicit binding. This is useful when you want to connect a transformation to a specific input or output port.

- An output of some other process.
- Something external, usually a file on disk.

For example, in "Implicit connections" you saw a pipeline that takes an input and produces an output. This is a simple example of implicit binding.

- An inline document in the XSL file. This is a template that is used to transform the input. This is a simple example of explicit binding.
- Nothing, which is useful in some cases.

Programming in XProc

X Porc is not a man just because it is not a full-fledged language.

Looping, branching and error handling

XPort is designed for interoperability (a, a brother in: gc h(o o s p : i) f a n d r e
handl ing) (functionality

XPath

Underlying the entire thing is a single thought that I have as a corner of a great big world is a corner of a great big world. Data is a part of a great big world.

Variables

You can always find computer algorithms, and even variable input sequences, but if you want to find out what the distribution of the attributes is, you need to collect data from the population of interest.

Liek XTS and Quercus XE is a functional programming language available as immutable.

Additional information

[illegible]

- The oXp rs t d a n d a s e l f e b b a h g u a g e c s p e c i f c a y) o n a a n d e
[h t t / p s : p / e o c x g 0 / r /](http://ps.ple.co.cxg0/r/) ([h t t s p e o c x g 0 / r /](http://st.ple.co.cxg0/r/))
- The r e a r t w o c X p o p e r s s o r s i n t h e m a k i n g :
 - XML C a l [lab / x m / i](http://lab.xml.it) (c a l a b a (s h t x o p d n a l a b) a s i h s . c t o l m e) c o p e r s
 p o c e s s o r e d e p e d a n d m a n i o t m a i n s e h . b
 - M o g a n a X p r t (t / p w w / w x m o j e c t . (c h o t n w p . w x / m o j e c t) . c i o s n) t h e c o p
 X P o r c o p e s s o r e d e p e d a n d m a n i t p i o e . t b b / e r o i o n i s o
 b a n d e d g a M a X P - r l l l " .

Both coprocessors are designed to complement XPr

- The author of this swamintirederacep rfoocr 3X.P0r. It will
XML eP s t (t/ p x m e p s . n e h t t/ x n e p s .) e b e g i n n i n g l o 2 o k 2 i v a d g o 1 6 y o r
p u c h r a s e ;)

- X Porc ives l d p e d o m t G / i g t i H t u h b u : b o c c o (n h / t x p o r t h u b o c c o) m B / y x p a r l l n
us! Discuss and s e p m o s y t t a i n s d e u n p a k d u l e s r t y l s , c a u s c r t h a , w e t
a p p c i a t e i t .
- J o i n t o h c e m X a P i r l i n t g t p l s i i : s t t g a l A B e s / r P u b o c e v d / e p r
(h t t p s i s t g / A B e s / r P u b o c e v d / e) p r

A t i c l e c o n t e n t s © 2019 E r i k S i e g e l

© **Textuality Services, Inc.** e x e p t f o r t i t c h l æ s s e w a i r t h n a m y e r d i g a h u t t h h o o r l s d e o l
t a r d e m a r k s g a d a d e m a r k s a p p e a r i n g t o o p e x M L o f e a n p e a r t r i v
o w n e r s .